

AUTONOMOUS FOOD RESCUE ENGINE

Feed the world. Not the bin.

AI-powered surplus prediction and real-time redistribution — connecting institutions to food banks before food ever hits the bin. Zero human coordination required.

See How It Works →

github.com/jarmancarrera/foodflow-ai

6 tools

CLAUDE AGENT LOOP

~4s

FULL RESCUE CYCLE

\$2.2B

TAM

380x

ENTERPRISE LTV:CAC

Claude claude-sonnet-4-6

FastAPI

SQLite

Python 3.11

Jinja2

reportlab

1 / 12

01 The Problem

Abundance and hunger coexist because of a timing failure, not a supply failure



SCALE OF THE CRISIS

40%

of the entire US food supply is wasted every year — enough to end domestic hunger 3x over

USDA Economic Research Service, 2023

\$444B

in economic value lost annually to food waste across the US supply chain

ReFED Roadmap to 2030, 2023

13.5%

of US households face food insecurity — 44M people, including 13M children

USDA ERS Household Food Security 2023

8–10%

of global greenhouse gas emissions come from food that's produced but never eaten

FAO The State of Food and Agriculture, 2023

WHY EXISTING SOLUTIONS FAIL



Manual phone calls

Staff calls food banks at 5:55 PM. No answer. Food goes to dumpster. This happens in 60% of attempted donations — timing is the failure mode, not intent.



Too Good To Go

Consumer-facing "magic bags." Requires retail packaging, consumer pickup, and staff time. Doesn't work for 200 lb hot-food surpluses at institutions.



Feeding America

Logistics network, not technology. Still requires human calls, scheduling, and coordination. No AI prediction or autonomous dispatch layer.



Liability fear

Institutions cite legal risk as their #1 reason not to donate. The Bill Emerson Act protects them — but nobody verifies compliance automatically. FoodFlow does.

02

The Solution

Claude claude-sonnet-4-6 tool_use agent — 6 tools, zero human coordination



FoodFlow connects to a restaurant's inventory system, detects surplus 2 hours before closing, finds the nearest food bank, locates a volunteer driver, verifies legal compliance, and dispatches the pickup — all in ~4 seconds, with zero staff involvement.

THE AUTONOMOUS 6-TOOL AGENT LOOP

TOOL 01
Inventory

Query

SELECT surplus items
WHERE expiry < 4h

Returns

Items, quantities, food type, expiry

Claude

Reads total lbs + food categories

TOOL 02
Food Bank

Query

Filter by capacity + food type match

Returns

Best bank, distance, available capacity

Claude

Picks closest with enough room

TOOL 03
Volunteers

Query

Haversine distance, available=true

Returns

Nearest driver, phone, capacity

Claude

Confirms vehicle can hold the load

TOOL 04
Route + ETA

Calc

Driver→Supplier distance

Returns

Total mi, ETA minutes, arrival time

Claude

Checks food won't expire en route

TOOL 05
Compliance

Check

Bill Emerson Act eligibility rules

Returns

is_compliant, liability_notes

Claude

Gates dispatch — blocks if fails

TOOL 06
Dispatch

Action

Log to DB, mock SMS to driver

Returns

pickup_id, status, timestamp

Claude

Confirms in natural language summary

Claude as Orchestrator
Not a script — Claude reasons across all 6 tools, handles edge cases in natural language, adapts when a food bank is full

Why ~4 Seconds?
Local SQLite = millisecond queries. Only latency is the Anthropic API call (1–2s). 6 tool calls x <100ms each = complete rescue in one coffee sip

3 Trigger Sources
Cron job 2h before closing · Manual dashboard button · POS webhook when surplus threshold crossed

03

Technical Architecture

Every file, every function, how they connect — full system diagram



LAYER 1 — FRONTEND

Dashboard UI `Jinja2`

`templates/dashboard.html`

Auto-refreshes every 10s via meta refresh. Shows live stats, surplus cards per location, trigger buttons, pickup log. Pure server-side render — no JS framework needed.

Styles

`static/style.css`

Custom CSS — dark green theme, card grid, responsive stat boxes, status badges.

LAYER 2 — API

FastAPI Application `main.py`

6 routes: GET / (dashboard), GET /api/stats, GET /api/surplus, POST /api/trigger/{id} → spawns BackgroundTask(run_agent), GET /api/pickups, GET /api/report. Uvicorn ASGI server.

LAYER 3 — DATA

SQLite Database

`database.py` · `foodflow.db`

Tables: food_inventory, food_banks, volunteers, pickups. Seed data: Cornell Statler Hall, RPCC, Ithaca Food Bank, 3 volunteer drivers. get_stats() / get_surplus_items() helpers.

LAYER 4 — AI AGENT CORE

Claude Agent Loop `agent.py`

run_agent(trigger) → while loop calling Anthropic API with TOOL_SCHEMAS → dispatches TOOL_FUNCTIONS[block.name](**block.input) → appends tool_result → repeats until stop_reason=="end_turn". Model: claude-sonnet-4-6, max_tokens: 4096.

Tool Functions + Schemas `tools.py`

TOOL_FUNCTIONS dict maps 6 tool names to Python callables. TOOL_SCHEMAS list is the JSON schema array passed to Anthropic API — defines input types, required fields, descriptions that Claude reads to understand each tool's capabilities.

LAYER 5 — REPORTING

ESG PDF Generator

`report.py`

reportlab PDF: cover page, impact stats table (lbs rescued, meals, CO₂ avoided), compliance log, trend data. Triggered via GET /api/report — auto-downloads PDF. Replaces \$500/quarter manual reporting.

Request Flow (one button click → rescue dispatched)

Browser → POST /api/trigger/statler → BackgroundTask → agent.py → Anthropic API → tools.py → database.py → dispatch logged → dashboard refresh

04

How the Agent Loop Works

agent.py line-by-line — explained for everyone



THE CORE CODE — ANNOTATED

```
# agent.py — FoodFlow's orchestration brain
def run_agent(trigger: dict) -> dict:
    messages = [{ "role": "user", "content": f"Rescue surplus at {trigger['location']}"
    }] # Loop until Claude says "I'm done"
    while True:
        response = client.messages.create(
            model="claude-sonnet-4-6",
            tools=TOOL_SCHEMAS, # Claude reads these to know what tools exist
            messages=messages ) # Claude is done — return its natural language summary
        if response.stop_reason == "end_turn":
            return {"status": "complete", "summary": response.content[0].text}
        # Claude wants to call a tool — execute it
        for block in response.content:
            if block.type == "tool_use":
                result = TOOL_FUNCTIONS[block.name](**block.input)
                # Send the result back so Claude can reason on it
                messages.append({
                    "role": "user",
                    "content": [{
                        "type": "tool_result",
                        "tool_use_id": block.id,
                        "content": json.dumps(result)
                    }]
                })
```

STEP-BY-STEP IN PLAIN ENGLISH

- 1 We send Claude a task**
"There's surplus food at Statler Hall. Please coordinate a rescue." — natural language, no specific instructions on HOW.
- 2 Claude decides to call a tool**
Instead of text, Claude returns: "I want to call check_inventory with location_id='statler'". This is the tool_use block.
- 3 We run the Python function**
TOOL_FUNCTIONS["check_inventory"](**{"location_id": "statler"}) queries SQLite, returns JSON with surplus items.
- 4 We give Claude the data**
We append tool_result to the conversation. Claude reads the JSON, reasons about it, and picks the next tool. Repeats 5 more times.
- 5 Claude returns stop_reason = "end_turn"**
After dispatch_pickup succeeds, Claude writes: "Dispatched Sarah Chen (0.8mi) to pick up 45lbs from Statler Hall. ETA: 12 min to Ithaca Food Bank."

Why This Beats Rule-Based Logic

A script needs hard-coded if-else for every edge case. Claude handles: "Food bank A is full → try B → B only takes cold food → our food is hot → escalate to human." That flexibility is impossible without LLM reasoning.

05

Market Sizing

TAM / SAM / SOM — bottom-up validated, conservative estimates

FoodFlow AI

YEAR 3 REVENUE BUILD (BOTTOM-UP)



Enterprise — Universities & Large Chains
\$2,500/mo · 15 customers · Cornell → Ivy → Big10 chain

15 customers

\$450K ARR



Growth — Mid-Market Restaurant Groups
\$299/mo · 20 customers · 5–50 location chains

20 customers

\$71.8K ARR

\$2.2B

TAM



Starter — SMB Restaurants

\$99/mo · 30 customers · 1–3 location operators

30 customers

\$35.6K ARR

All US food institutions :
500K restaurants +
4,700 universities

+ Success fees (2% food value rescued)

+ \$63.6K ARR

+ ESG Reports (\$500/quarter × 10 customers)

+ \$20K ARR

Total ARR Year 3 (incl. 115% NRR)

Conservative — only 0.43% of SAM penetration required

\$1.82M

\$1.82M

SOM

Year 3

\$426M

SAM

Urban universities +
restaurant groups in top
50 metros

06

Business Model

Three revenue streams - SaaS + success fees + ESG reports

FoodFlow AI

STARTER

\$99 /mo

1 location - ideal for independent restaurants and small operators ready to start rescuing

- ✓ 1 location monitored
- ✓ Real-time dashboard
- ✓ Basic ESG report (annual)
- ✓ Email alerts

✗ API access

GROWTH

\$299 /mo

Up to 5 locations - for restaurant groups and mid-size campus dining operations

- ✓ 5 locations monitored
- ✓ API & webhook access
- ✓ Monthly ESG reports
- ✓ Custom alert rules
- ✓ Priority support

MOST POPULAR

ENTERPRISE

\$2,500 /mo

Unlimited locations - for universities, hospital networks, and large restaurant chains

- ✓ Unlimited locations
- ✓ White-label branding
- ✓ Quarterly board-ready ESG deck
- ✓ Dedicated CSM
- ✓ POS integration support



SaaS Subscription

Monthly recurring — 100% predictable. Anchors unit economics. \$99–\$2,500/mo.

78% of revenue



Success Fee (2%)

2% of food market value rescued. Enterprise: ~\$6,570/yr. Scales with usage.

12% of revenue



ESG Report (\$500/quarter)

Replaces \$5K–\$15K consulting engagement. GRI 306, SDG mapping, Scope 3 data.

10% of revenue

78% Gross Margin

380x LTV:CAC (Enterprise)

8 days CAC Payback

115% NRR Target

\$288 Enterprise CAC

07

Unit Economics

Enterprise tier deep-dive — 78% gross margin explained

FoodFlow AI

\$3,047

ARPA / month (Enterprise)
SaaS + fees + ESG

✓ HEALTHY

\$109,700

LTV — 36 months
at 3% monthly churn

✓ HEALTHY

\$288

CAC — founder-led
warm intro sales

✓ EXCELLENT

8 days

CAC payback period
(\$288 + \$2,375 GM/mo)

✓ EXCELLENT

GROSS MARGIN WATERFALL — ENTERPRISE \$2,500/MO



COGS BREAKDOWN — ENTERPRISE (% OF \$2,500 REVENUE)



⚡ LTV:CAC of 380x — how is this possible?

Enterprise buyers (university sustainability directors) are pre-sold on the concept — they have ESG mandates and legal pressure to act. Sales is a warm intro from a peer, not a cold outbound call. One Zoom demo + a Cornell case study = signed. CAC is literally the cost of founder time for one meeting. LTV is 36 months × \$2,375 gross margin = \$85,500 minimum.

08

Go-to-Market & Competitive Moat

Cornell Wedge → Ivy Chain → National

FoodFlow AI

THE CORNELL WEDGE STRATEGY

MONTH 1-2

3 Free Design Partners on Campus

Statler Hotel, RPCC, + 1 local restaurant. Free deployment. Collect real rescue data, build a case study with actual lbs rescued and CO₂ numbers.

\$0 MRR → \$0 CAC → priceless case study

MONTH 3-4

Cornell Dining Full Contract

Convert design partners. Cornell Dining runs 8 halls → \$2,500/mo × 8 = one signed deal = \$20K MRR. First real revenue.

\$20K MRR · Payback: 8 days

MONTH 5-6

Ivy League Referral Chain

Cornell sustainability director introduces to Harvard, Penn, Columbia. Warm intro = \$288 CAC. Close 3 more universities from one coffee meeting.

\$30K MRR · \$360K ARR run rate

MONTH 7-18

Restaurant Groups + \$500K Pre-Seed

University case studies unlock restaurant group sales. Same ESG pressure, same buyer. Raise \$500K to hire first SDR + backend engineer.

\$1.82M ARR by Month 36

COMPETITIVE MATRIX

FEATURE	FOODFLOW	TGTG	COPIA	MANUAL
 FoodFlow AI	—	—	—	—
Autonomous AI dispatch	✓	✗	✗	✗
Institutional-grade (bulk)	✓	✗	✓	±
Zero staff time required	✓	✗	✗	✗
ESG report auto-generation	✓	✗	±	✗
Bill Emerson compliance check	✓	✗	±	✗
LLM edge-case reasoning	✓	✗	✗	✗



The Data Moat

After 6 months at one institution, FoodFlow knows: "Statler Hall always has 40–60 lbs of hot food on Fridays." Predictions become self-improving. This training data is impossible to replicate without the deployment history — and it grows with every rescue.



10

Social Impact & ESG Value

The reason institutions pay — sustainability mandates, not charity



FoodFlow doesn't ask institutions to be charitable. It gives them a **compliance tool** that happens to rescue food — and auto-generates the ESG report their board requires.

12M

lbs food rescued annually by Year 3

7.2M

meals served to food-insecure families

8,400

metric tons CO₂ avoided per year

65

institutions across 3 US cities

2

SDG 2 — Zero Hunger

Directly addresses food insecurity by redirecting surplus to food banks; 7.2M meals = measurable SDG progress.

12

SDG 12 — Responsible Consumption

Reduces institutional food waste at source. Institutions get GRI 306 disclosure data automatically generated.

13

SDG 13 — Climate Action

8,400 MT CO₂ avoided. Feeds directly into Scope 3 emission reduction reporting for SEC ESG disclosures (2026).

17

SDG 17 — Partnerships

Builds institutional-to-foodbank partnership infrastructure that grows with every rescue and every new partner added.

11

The Ask — \$50K Seed

Working MVP live today · Cornell pilot ready · Ivy chain waiting

FoodFlow AI

SEED ROUND ASK

\$50,000

Pre-seed angel round · 18 months runway · breakeven by Month 6 on first 3 enterprise contracts

	Product: Twilio SMS + POS integrations	\$15K	30%
	Sales: Cornell pilot + 3 Ivy intros	\$12K	24%
	Infra: cloud hosting + API costs	\$8K	16%
	Legal: entity + Bill Emerson docs	\$5K	10%
	Operating reserve (6 months)	\$10K	20%

WHAT'S BUILT TODAY (APRIL 25, 2026)

- ✓ Working Claude tool_use agent — 6 tools, full loop
- ✓ FastAPI dashboard — real-time, auto-refresh 10s
- ✓ ESG PDF auto-generator via reportlab
- ✓ Cornell seed data — Statler, RPCC, Ithaca Food Bank
- ✓ Open source: github.com/jarmancarrera/foodflow-ai

RUN IT NOW

```
# Clone + run in 3 commands git clone git@github.com:jarmancarrera/foodflow-ai.git export ANTHROPIC_API_KEY=sk-ant-... uvicorn main:app --reload --port 8000 # -> http://localhost:8000 -> click Rescue Now
```

FoodFlow AI

Cornell CBC Hackathon · April 25, 2026

github.com/jarmancarrera/foodflow-ai
jarmancarrera@gmail.com